SOFTWARE ENGINEERING PROJECT

TOPIC:

DISASTER MANAGEMENT AND RELIEF SYSTEM

SUBMITTED TO:

MS. ASIA SAHAB

SUBMITTED BY:

HERMAIN KASHIF 240901026

SYEDA ZAINAB RAZA 240901035

EIRAJ FAISAL 240901038

Contents

Part completed by Syeda Zainab raza, Hermain kashif, Eiraj Faisal (35, 26, 38)

INTRODUCTION:

When a disaster strikes, countless number of people suffer from the consequences. There is a large amount of damage to human resources in these cases. In scenarios like these, not receiving proper help can be a major issue for the affected victims. To solve this problem, there must be a proper connection between NGO's, volunteers, donors, and victims.

The disaster management and relief system helps solve all these problems promptly. This system helps connect all the participating NGO's, volunteers, and donors to the victims through a centralized system. This system accepts complaints and help requests from disaster victims and passes them on to individuals willing to assist. These people then accept the request and contribute where needed the most.

This system helps ensure all the activities are transparent, well-organized, and efficient. This system also helps the government to keep track of available resources to reduce confusion and malpractice during emergencies.

The main goal of this project is to efficiently provide help and relief to the people in need so they can recover fast from the damages they faced.

Part completed by Syeda Zainab raza (240901035)

FUNCTIONAL REQUIREMENTS:

1: Volunteer registration:

- The volunteers must register themselves with their names and personal information.
- The user must add information about their role in the system.
- Every user must have their designated login ID.

2: Complaint management center:

- The user can enter their complaints in the system complaint center.
- The victims can mention their needs and wants in case of any disaster situation.
- The system must receive and process the complaint and forwards it to the NGO'S and volunteers for them to take action.

3: Assigning tasks to volunteers:

- The complaint of the victim is processed by the system.
- The system shows the complaint on the user's side.
- The system assigns the complaint to the available volunteers by tracking the volunteer activity.
- The volunteers who are assigned the task then take action to resolve the issue.

4: Management of resources:

- The system keeps track of the resources that are available for the victims.
- The authorized personnel can also alter the records by entering or deleting any extra information.
- They can enter the any updates in the donation about any new donations.

5: Track of volunteer activities:

- The system keeps track of the activities of different volunteers.
- It checks if the volunteers are available.
- It shows what volunteers are assigned to what task at what time.
- The volunteers can add and update their status according to their availability.

6: Checking shelters and relief camps:

- The system keeps a track of the availability of shelter camps for the victims in case of a disaster.
- The volunteers can also update the availability or unavailability of the shelters.
- The system can also keep track of the location of the relief camps
- It can also track the amount of people a relief camp can accommodate.

7: Disaster alert system:

- The system helps the volunteers can get a warning message or an alert in case of any kind of emergency situation.
- This will help the volunteers to take an immediate action upon the situation and prepare necessary reliefs for the affected people.
- The alerts are traced so the system can inform the ngo/volunteers about the location.

8: Generation of reports:

- The system is able generated well written reports for the user.
- These reports may contain information about a specific disaster.
- These reports can be about the number of resources used in a specific situation or at a specific location.
- These reports help the users to keep track of the relief management they carried out for the victims.

9: Handling system without internet:

- This system helps manage and helps enter the information in the system without the need of internet.
- The system and the information will synchronize as soon as the system catches any signal of internet.

10: Security:

- The system must provide full confidentiality to the victims and members.
- The donors must not be able to access any information about the victims.
- Only the managers must be able to view everything about the victims, donors, NGO's etc.

Part completed by Eiraj Faisal (240901038)

NON-FUNTIONAL REQUIREMENTS:

1: Performance requirements:

- The system should be able to load within five seconds.
- The system maps and volunteer activity should be updated every 15 minutes to keep the system updated about their availability.
- The system should be able to handle a lot of traffic at the same time in case of any emergency.

2: Security Requirements:

- The system should keep the confidential information about the volunteers, etc., restricted to the authorized people only.
- The system should also protect the information about the resource management from itself.

3: Usability Requirements:

- The system should be usable, which means it should be user-friendly.
- The system is to be used by many users, such as victims, volunteers, and Donors, so the user interface should be easy to use and should not be complex so that even a layman can easily figure it out.

4: Scalability Requirements:

- This system should be scalable and should be able to handle thousands of requests at a time without lagging.
- Scalability is an important factor for any system. A system that lacks it is very inefficient. So, this requirement should be fulfilled.

5: Maintainability Requirements:

- The system should be maintainable and updatable.
- Any new updates should be accessible easily, and there should be room for maintenance to increase system efficiency.

6: Reliability Requirements:

- The system should be reliable to its users, meaning that the victims can rely on the system and be sure that the help they need will reach them.
- Whereas Donors can rely on the system, that their donations will surely reach the needy.

7: Availability Requirements:

- The system should be available to its users at all times.
- Even without internet, the users can use the system

Part completed by Syeda Zainab raza (240901035)

UPPER USE CASE:

1: Volunteer registration

Use-case body	Volunteer Registration
Trigger	When a volunteer wants to register in the system.
Pre-condition	Volunteer opens the registration page.
Basic Path	1. Volunteer selects the option 'Register as a volunteer'. 2. The system asks if the volunteer is part of an NGO or not. If yes, then the NGO name is required. 3. Then inserts personal info (name, email, etc.). 4. Volunteer selects the field he wants to volunteer in (medical, rescue operation, etc.). 5. Volunteer: add username and password. 6. The system verifies the volunteer by sending a verification message. 7. Admin approves the volunteer based on experience and history. 8. The system sends confirmation to the volunteer.
Alternative Path	If the system enables auto-approval, the volunteer is added to the database.
Post condition	Volunteer record is added to the database.
Actors	Volunteers, System
Exception Paths	If the volunteer leaves any important info unentered during registration, they cannot proceed.
Others	Volunteers can update profiles

2: Complaint Management Center

Use-case name	Complaint Management Center
Trigger	The system helps the victims to submit help requests in case of emergencies.
Pre-condition	The victims must be entering their location.
Basic Path	1: The victims enter the system and open up the complaint center. 2: The victims type their complaint. 3: The victims select the intensity of their complaint and their location. 4: The victims select the type of help required. 5: The system takes the complaint and further processes it to different NGO's and volunteers etc.
Alternative paths	If the complaint is not sent, the system tries to find a better connection to process the complaint.
Post Condition	The complaint is sent further to the system and then the system forwards the complaint to the designated admin to take action.
Actors	Victims, NGO/volunteers, System
Exception Paths	Invalid or wrong information gets rejected.
Others	The system ensures that the complaint is processed.

3: Assigning tasks to volunteers

Use-case name	Assigning of tasks to volunteers
Trigger	A system handler assigns a victim request to a volunteer.
Pre-condition	There should be a victim request in the system for the system to assign to the volunteer.
Basic Path	1: The system processes the victim request. 2: The manager assigns a victim request to the available volunteer. 3: The volunteer accepts the task from the system manager. 4: The system updates the availability of the volunteer. 5: The system changes the status of the complaint.
Alternative paths	If one volunteer becomes unavailable, the system manager can assign the complaint to another volunteer.
Post Condition	The complained is marked as “complain being handled” or “complain handled successfully.
Actors	System, Volunteers
Exception Paths	If there is an unavailability of volunteers or an invalid complaint.
Others	The system handles a victims problem successfully.

4: Management of resources

Use-case name	Management of resources
Trigger	The system helps manage keep a track of the number of resources available for the victims collectively.
Pre-condition	The NGO’s and volunteers must give resources to the system.
Basic Path	1: The NGO’s and volunteers give their donations and enter the records into the system. 2: They can add and delete records from the system. 3: System gets updated incase of any donations from the system. 4: The updates appear on the system center.
Alternative paths	Resources can be donated by anonymous personnel with proper registration.
Post Condition	Any additions or subtractions in the system are updates.
Actors	Donors, System, Volunteers
Exception Paths	If the system has invalid entries.
Others	The system updates the records in accordance with different NGO’s and volunteers. NGO’s and volunteers can only update their own personal records.

5: Track of Volunteer activities

Use-case name	Track of volunteer activities
Trigger	The measures and steps taken by organizations and volunteers in case of any disaster or emergency.
Pre-condition	The volunteer gets assigned a task.
Basic Path	1: The volunteer gets assigned a task in case of any emergency by the system. 2: The volunteer takes action on completing the assigned task. 3: If the task is in process the it gives “in process”. 4: The system updates the activity of the volunteer to “Completed” when the task is completed.
Alternative paths	If the volunteer has any problem in completing the task, the system can assign the task to any other available team.
Post Condition	The system updates the volunteer status as the task is completed.
Actors	System, Volunteers/NGO’s
Exception Paths	The system is not able to update the status on time.
Others	The system is updated only when the confirmation from both the volunteer and victim is received.

6: Checking shelters and relief camps

Use-case name	Checking shelters and relief camps
Trigger	Building shelters and camps for victims who loose their homes and adding the information into the system.
Pre-condition	The system should have the information about the availability of the shelter.
Basic Path	1: The system can add more shelters if there is a shelter made by any volunteer. 2: The volunteer is going to add the information about the shelter along with its location. 3:The system will update or add shelter in the system.
Alternative paths	The previous shelters can be reused in case there is an unavailability of shelters.
Post Condition	The data will be updated according to the changes.
Actors	System, Volunteers/NGO’s
Exception paths	Invalid data or wrong location provided.
Others	Information of the availability of shelters will be updated often. The system should also provide the information about how many people can a shelter accommodate.

7: Disaster Alert system

Use-case name	Disaster alert system
Trigger	There will be an alert or warning message in case of any disaster for the volunteers.
Pre-condition	There is any new complaint, news or prediction of the disaster.
Basic Path	1: The system throws a warning message in case of any threat. 2: The system generates an alert message for the volunteers in case of active emergency.
Alternative paths	The volunteer can reject an emergency call.
Post Condition	The messages are successfully delivered to the users of the system.
Actors	System, Volunteers/NGO's
Exception Paths	There is a false warning or message. The warning failed to process.
Others	The alerts should be tracked and marked.

8: Generation of reports

Use-case name	Generation of reports
Trigger	The user of the system asks for a report.
Pre-condition	The user must be a registered volunteer.
Basic Path	1: The user asks he system to generate a report for a specific disaster event on a specified date. 2: The system will gather the information about the victims, resources or volunteers. 3: The system will generate the report for the user.
Alternative paths	Filter according to the region.
Post Condition	The system generates a report and the user has a saved report.
Actors	System
Exception Paths	Inconsistent data can cause wrong reports.
Others	There can be separate reports for each organization.

9: Handling system offline/ without internet

Use-case name	System handling without internet
Trigger	Allows the user to enter data without internet.
Pre-condition	The user should be able to open the system.
Basic Path	1: The user enters their information. 2: The system saves the information. 3: The system is not updated because of the unavailability of internet. 4: The system syncs the newly entered information after it gets a connection.
Alternative paths	User can trigger the synchronization manually too.
Post Condition	The system is updated as soon as there is an availability of internet.
Actors	System, Victims
Exception Paths	The file does not get in the process of sending.
Others	The system can display the phase the file lies in during the sending process.

10: Security and confidentiality

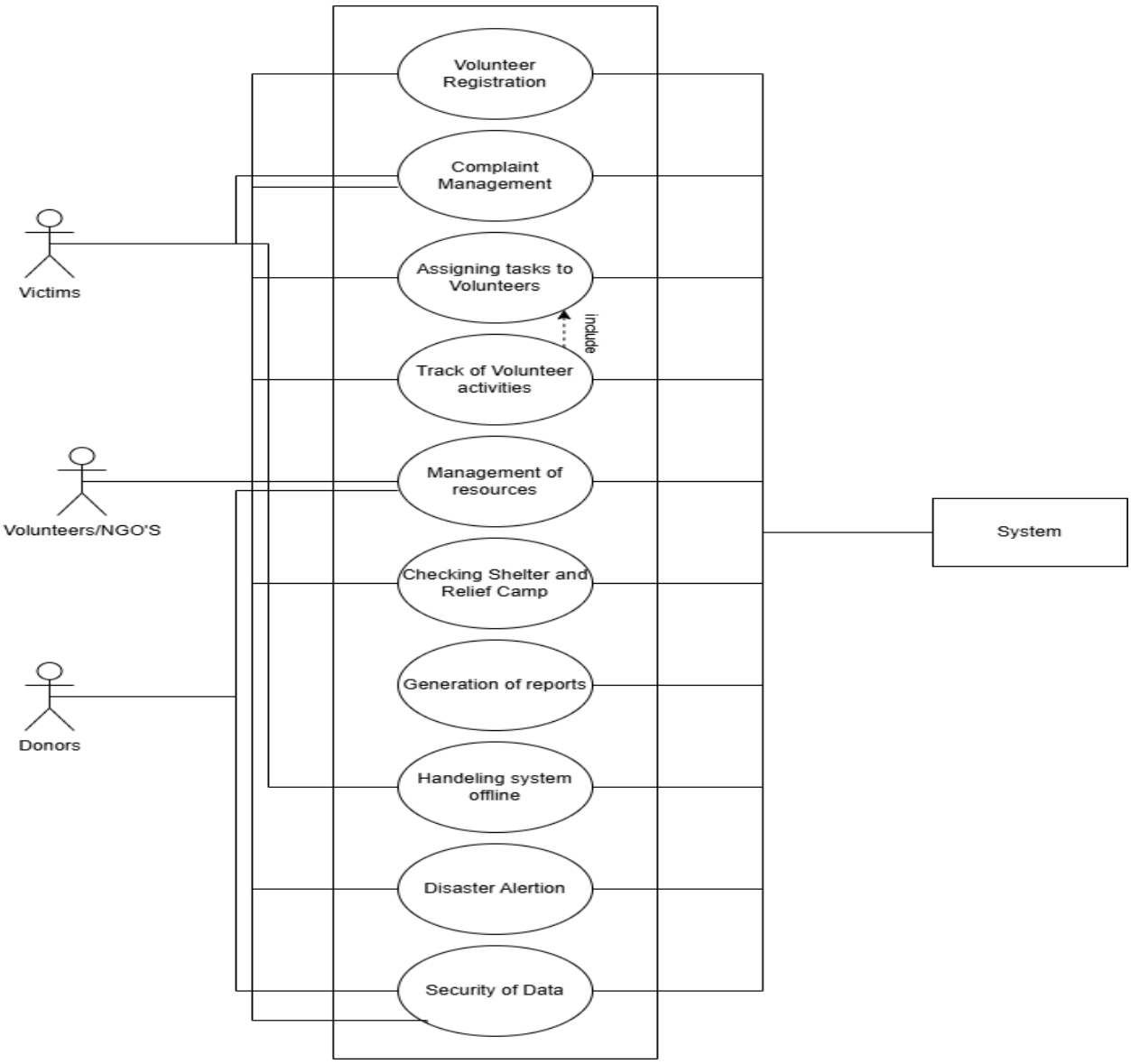
Use-case name	System security and confidentiality
Trigger	The user tries to login
Pre-condition	The user must have a proper login info
Basic Path	1: The user enters the system. 2: The system asks the user to verify his identity. 3: The user enters his role. 4: The system recognizes the role. 5: The system asks the user for their name and assigned password. 6: The user can enter the system and access the information allowed by the system.
Alternative paths	The system will warn the user if the wrong password is entered.
Post Condition	The system successfully allows the user to access their desired information.
Actors	System, Volunteers/NGO's, Donors
Exception Paths	If the user doesn't know the password, they cannot enter the system.
Others	The user can change their password by following a proper procedure in case they forget their password.

Part completed by hermain Kashif (240901026)

Diagrams:

UML Diagram

Unified Modeling Language (UML) diagrams are used in this project to visually represent the structure, behavior, and interactions of the Disaster Management and Relief System. These diagrams help clarify the system design and provide a structured way to document how different components of the system work together.


Volunteer Registration
Complaint Management
Assigning tasks to Volunteers
include
Track of Volunteer activities
Management of resources
Checking Shelter and Relief Camp
Generation of reports
Handeling system offline
Disaster Alertion
Security of Data
Victims
Volunteers/NGO'S
Donors
System

CLASS DIAGRAM

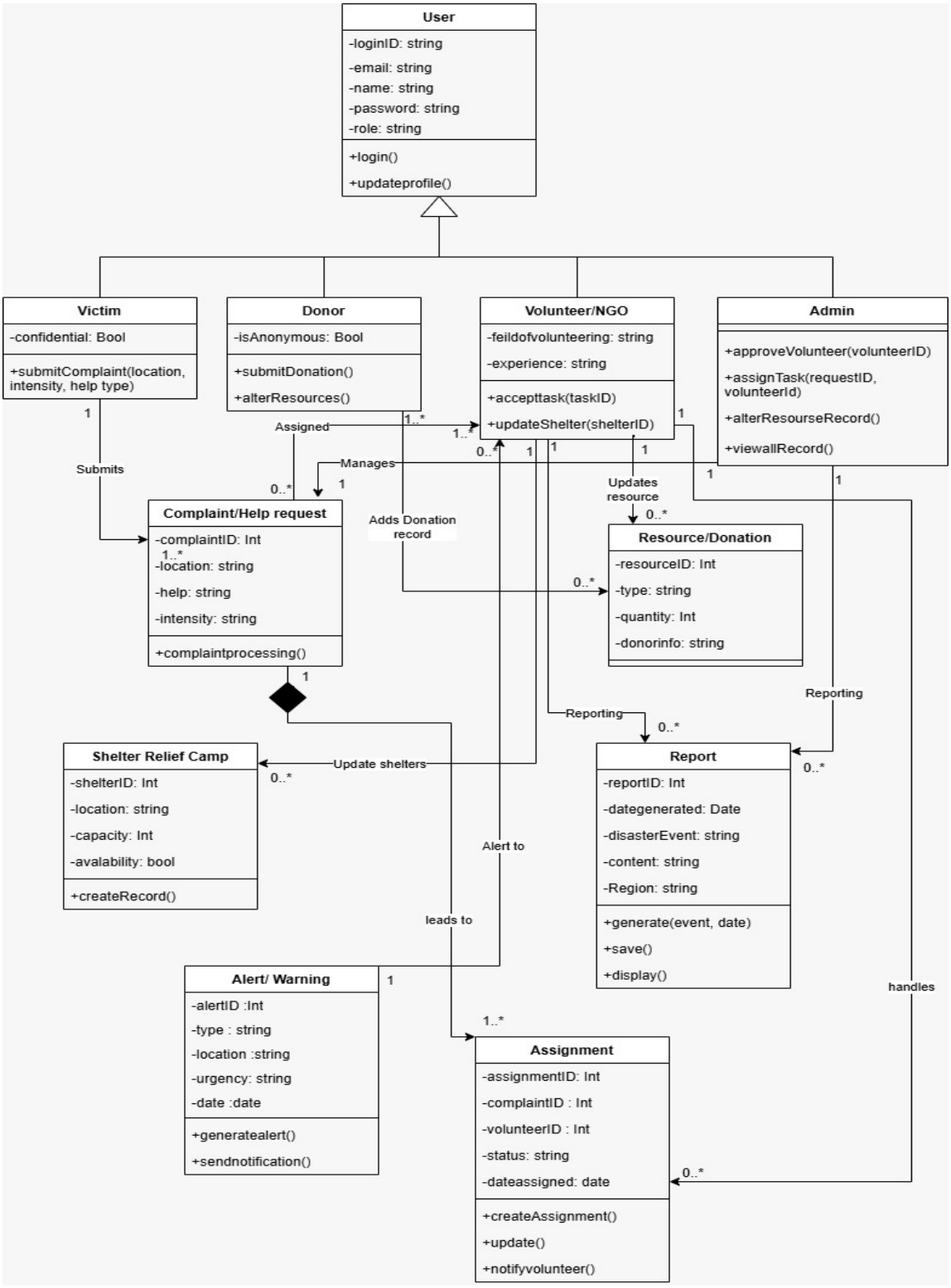
• **User → Victim:** Inherits from User (**inheritance**).
• **User → Donor:** Inherits from User (**inheritance**).
• **User → Volunteer/NGO:** Inherits from User (**inheritance**).

• **Victim → Complaint:** One victim can submit many complaints (**one-to-many**).
• **Complaint → Assignment:** One complaint can generate multiple assignments (**one-to-many**).
• **Volunteer → Assignment:** One volunteer can handle many assignments (**one-to-many**).

• **Donor → Resource/Donation:** One donor can contribute multiple donations (**one-to-many**).
• **Volunteer → Resource/Donation:** Volunteers update resource records (**association**).
• **Volunteer → Shelter Relief Camp:** Volunteers update shelter information (**association**).

• **Alert → Volunteer:** One alert can notify multiple volunteers (**one-to-many**).
• **Complaint ↔ Volunteer (via Assignment):** Assignment connects a complaint to a volunteer (**linking relationship**).

• **Admin → Volunteer/Assignment/Resource/Report:** Admin supervises system operations (**association**).
• **Report → System Entities:** Report gathers data from complaints, assignments, resources, and alerts (**dependency**).


User
-loginID: string
-email: string
-name: string
-password: string
-role: string
+login()
+updateprofile()
Victim
-confidential: Bool
+submitComplaint(location, intensity, help type)
Donor
-isAnonymous: Bool
+submitDonation()
+alterResources()
Volunteer/NGO
-feildofvolunteering: string
-experience: string
+accepttask(taskID)
+updateShelter(shelterID)
Admin
+approveVolunteer(volunteerID)
+assignTask(requestID, volunteerId)
+alterResourseRecord()
+viewallRecord()
Submits
Assigned
Manages
Adds Donation record
Updates resource
Complaint/Help request
-complaintID: Int
-location: string
-help: string
-intensity: string
+complaintprocessing()
Resource/Donation
-resourceID: Int
-type: string
-quantity: Int
-donorinfo: string
Reporting
Shelter Relief Camp
-shelterID: Int
-location: string
-capacity: Int
-avalability: bool
+createRecord()
Update shelters
Report
-reportID: Int
-dategenerated: Date
-disasterEvent: string
-content: string
-Region: string
+generate(event, date)
+save()
+display()
Alert to
leads to
Alert/ Warning
-alertID :Int
-type : string
-location :string
-urgency: string
-date :date
+generatealert()
+sendnotification()
Assignment
-assignmentID: Int
-complaintID : Int
-volunteerID : Int
-status: string
-dateassigned: date
+createAssignment()
+update()
+notifyvolunteer()
handles

Code from Class Diagram

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

using Date = string;

class User;
class Victim;
class Donor;
class VolunteerNGO;
class Admin;
class ComplaintHelpRequest;
class ResourceDonation;
class ShelterReliefCamp;
class AlertWarning;
class Report;
class Assignment;

class User {
private:
    string loginID;
    string email;
    string name;
    string password;
    string role;
```

```
public:
    User(string id, string mail, string n, string pass, string r)
        : loginID(id), email(mail), name(n), password(pass), role(r) {}

    virtual ~User() {} // Virtual destructor for inheritance

    void login() {
        cout << "User " << name << " logged in." << endl;
    }

    void updateprofile() {
        cout << "Profile updated." << endl;
    }
};

class Assignment {
private:
    int assignmentID;
    int complaintID;
    int volunteerID;
    string status;
    Date dateassigned;

public:
    Assignment(int id, int cID, int vID, string stat, Date date)
        : assignmentID(id), complaintID(cID), volunteerID(vID), status(stat), dateassigned(date) {}
```

```
    void createAssignment() {
        cout << "Assignment created." << endl;
    }

    void update() {
        cout << "Assignment updated." << endl;
    }

    void notifyvolunteer() {
        cout << "Volunteer notified." << endl;
    }
};

class ShelterReliefCamp {
private:
    int shelterID;
    string location;
    int capacity;
    bool avalability;

public:
    ShelterReliefCamp(int id, string loc, int cap, bool avail)
        : shelterID(id), location(loc), capacity(cap), avalability(avail) {}

    void createRecord() {
        cout << "Shelter record created." << endl;
    }
};
```

```
class ResourceDonation {
private:
    int resourceID;
    string type;
    int quantity;
    string donorinfo;

public:
    ResourceDonation(int id, string t, int q, string dInfo)
        : resourceID(id), type(t), quantity(q), donorinfo(dInfo) {}
};

class ComplaintHelpRequest {
private:
    int complaintID;
    string location;
    string help;
    string intensity;

    vector<Assignment> assignments;

public:
    ComplaintHelpRequest(int id, string loc, string h, string intent)
        : complaintID(id), location(loc), help(h), intensity(intent) {}

    void complaintprocessing() {
        cout << "Processing complaint " << complaintID << endl;
```

```
    }

    void addAssignment(const Assignment& assign) {
        assignments.push_back(assign);
    }
};

class AlertWarning {
private:
    int alertID;
    string type;
    string location;
    string urgency;
    Date date;

    vector<ComplaintHelpRequest*> resultingComplaints;

public:
    AlertWarning(int id, string t, string loc, string urg, Date d)
        : alertID(id), type(t), location(loc), urgency(urg), date(d) {}

    void generatealert() {
        cout << "Alert generated." << endl;
    }

    void sendnotification() {
        cout << "Notification sent." << endl;
    }
```

```
};

class Report {
private:
    int reportID;
    Date dategenerated;
    string disasterEvent;
    string content;
    string Region;

public:
    Report(int id, Date date, string event, string cont, string reg)
        : reportID(id), dategenerated(date), disasterEvent(event), content(cont), Region(reg) {}

    void generate(string event, Date date) {
        cout << "Report generated for " << event << endl;
    }

    void save() {
        cout << "Report saved." << endl;
    }

    void display() {
        cout << "Displaying report content..." << endl;
    }
};

class Victim : public User {
```

```
private:
    bool confidential;

    vector<ComplaintHelpRequest*> myComplaints;

public:
    Victim(string id, string mail, string n, string pass, string r, bool conf)
        : User(id, mail, n, pass, r), confidential(conf) {}

    void submitComplaint(string location, string intensity, string helpType) {
        cout << "Complaint submitted by victim." << endl;

    }
};

class Donor : public User {
private:
    bool isAnonymous;
    // Association: Assigned to Complaint (1..* to 0..*)
    vector<ComplaintHelpRequest*> assignedComplaints;

public:
    Donor(string id, string mail, string n, string pass, string r, bool anon)
        : User(id, mail, n, pass, r), isAnonymous(anon) {}

    void submitDonation() {
        cout << "Donation submitted." << endl;
    }
```

```
    void alterResources() {
        cout << "Resources altered." << endl;
    }
};

class VolunteerNGO : public User {
private:
    string feildofvolunteering;
    string experience;

    vector<ComplaintHelpRequest*> managedComplaints; // Manages
    vector<ShelterReliefCamp*> shelters;            // Updates shelters
    vector<ResourceDonation*> addedDonations;       // Adds Donation record
    vector<Report*> reports;                        // Reporting
    vector<AlertWarning*> alerts;                   // Alert to

public:
    VolunteerNGO(string id, string mail, string n, string pass, string r, string field, string exp)
        : User(id, mail, n, pass, r), feildofvolunteering(field), experience(exp) {}

    void accepttask(int taskID) {
        cout << "Task " << taskID << " accepted." << endl;
    }

    void updateShelter(int shelterID) {
        cout << "Shelter " << shelterID << " updated." << endl;
    }
```

```
};

class Admin : public User {
private:
    vector<Assignment*> handledAssignments;  // handles
    vector<Report*> reports;                 // Reporting
    vector<ResourceDonation*> resources;     // Updates resource

public:
    Admin(string id, string mail, string n, string pass, string r)
        : User(id, mail, n, pass, r) {}

    void approveVolunteer(int volunteerID) {
        cout << "Volunteer " << volunteerID << " approved." << endl;
    }

    void assignTask(int requestID, int volunteerID) {
        cout << "Task assigned to volunteer " << volunteerID << "." << endl;
    }

    void alterResourseRecord() {
        cout << "Resource record altered." << endl;
    }

    void viewallRecord() {
        cout << "Viewing all records..." << endl;
    }
};
```

```
int main() {
    Admin admin("admin01", "admin@sys.com", "Super Admin", "securepass", "Administrator");
    admin.login();
    admin.viewallRecord();

    return 0;
}
```

DOMAIN MODEL DIAGRAM

- **Victim:** Submits **Complaint/Help Request** entries (location, type, intensity). A Victim can submit multiple complaints.
- **Volunteer/NGO:** Performs relief tasks, manages **Shelter Relief Camps**, and updates **Resource/Donation** availability.
- **Donor:** Provides financial or material support recorded in the **Resource/Donation** entity.
- **Admin:** A distinct authority figure with elevated permissions. Responsibilities include approving volunteers, assigning tasks, monitoring resources, and generating **Report**s.

Work Flow:

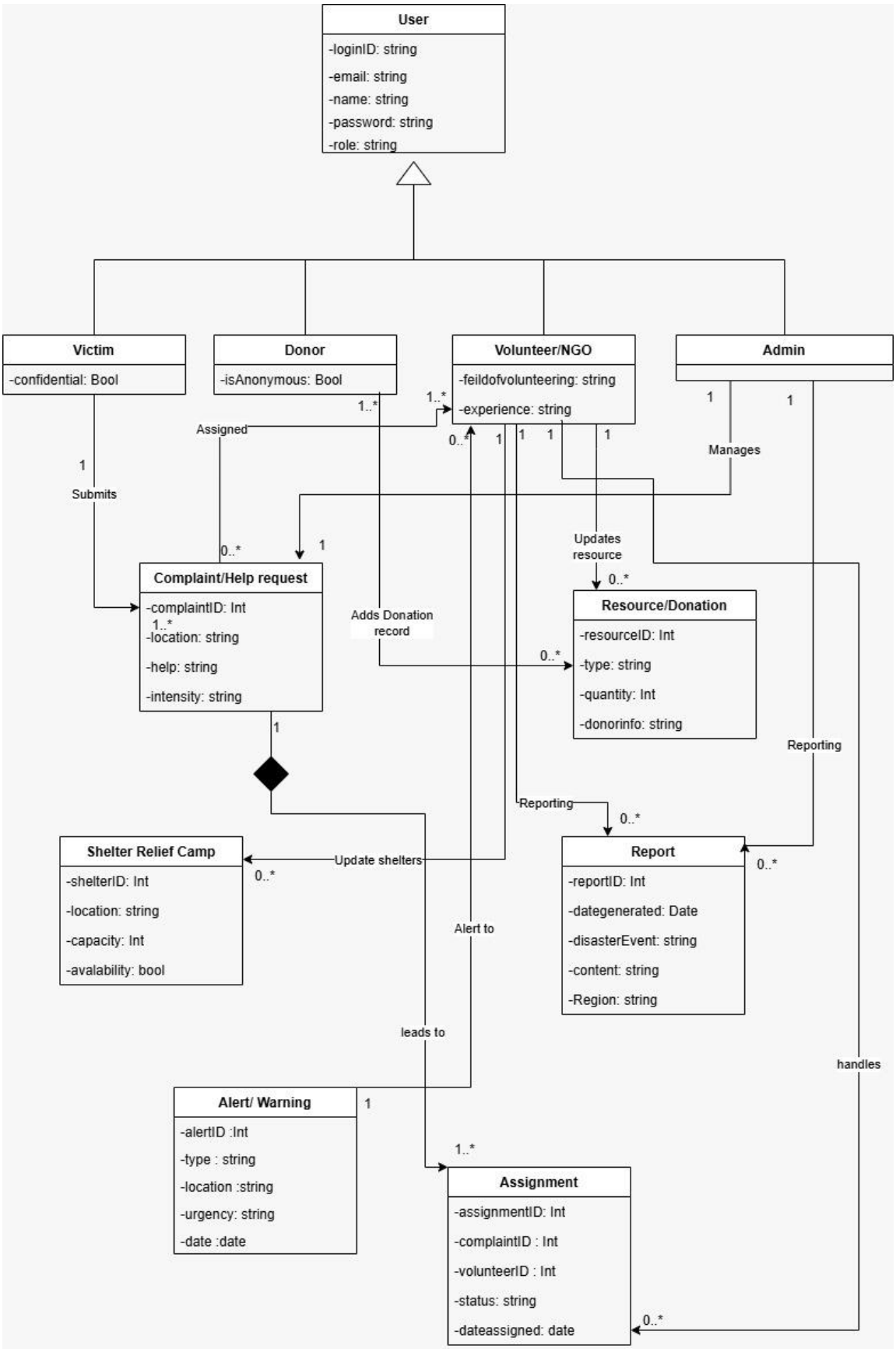
Complaint/Help Requests lead to one or more **Assignment**s .This entity links victims' needs directly to volunteers' actions.

Resource/Donation: Tracks all supplies ,updated by both Donors and Volunteers.

Shelter Relief Camp: Records temporary shelter details ,updated by Volunteers.

Alerts/Warnings: Notifies Volunteers about urgent conditions (type, urgency, location) to facilitate immediate action.

Report: Provides summary information for administrative monitoring of efforts, resources, and system performance


User
-loginID: string
-email: string
-name: string
-password: string
-role: string
Victim
-confidential: Bool
Donor
-isAnonymous: Bool
Volunteer/NGO
-feildofvolunteering: string
-experience: string
Admin
Assigned
Manages
Submits
Complaint/Help request
-complaintID: Int
-location: string
-help: string
-intensity: string
Adds Donation record
Updates resource
Resource/Donation
-resourceID: Int
-type: string
-quantity: Int
-donorinfo: string
Reporting
Shelter Relief Camp
-shelterID: Int
-location: string
-capacity: Int
-avalability: bool
Update shelters
Report
-reportID: Int
-dategenerated: Date
-disasterEvent: string
-content: string
-Region: string
Alert to
leads to
handles
Alert/ Warning
-alertID :Int
-type : string
-location :string
-urgency: string
-date :date
Assignment
-assignmentID: Int
-complaintID : Int
-volunteerID : Int
-status: string
-dateassigned: date

SEQUENCE DIAGRAM

- **Volunteer Registration:** A **Volunteer** submits details. The **System** validates the input, and the **Admin** approves the data, granting the volunteer system access.

- **Complaint Submission:** A **Victim** submits a complaint (location, type, intensity). The **System** processes the request and generates a unique ID.

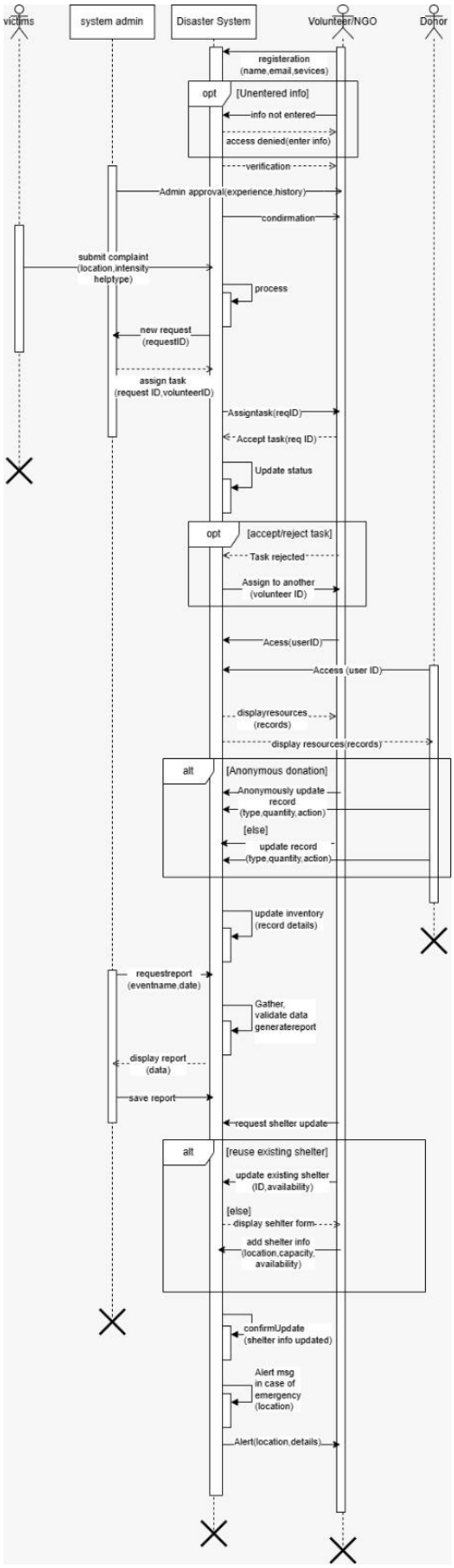
- **Task Assignment:** The **Admin/System** assigns the complaint to a **Volunteer**. If rejected, the system reassigns it. Upon acceptance, the volunteer's status is updated.

- **Resource Management: Volunteers, Donors, or Admins** update the resource database. The **System** records donations (type, quantity) and updates resource availability.

- **Shelter Management: Volunteers** update or create **Shelter** records. The **System** checks for reuse possibilities and updates location, capacity, and availability information.

- **Report Generation:** A user requests a report. The **System** gathers all necessary data, composes the report, presents it to the user, and saves it for documentation.

- **Alert/Warning Generation:** In an emergency, the **System** generates and sends an alert message (location, threat details) to **Volunteers** to enable prompt response.


victims
system admin
Disaster System
Volunteer/NGO
Donor
registeration (name,email,sevices)
opt
[Unentered info]
info not entered
access denied(enter info)
verification
Admin approval(experience,history)
condirmation
submit complaint (location,intensity helptype)
process
new request (requestID)
assign task (request ID,volunteerID)
Assigntask(reqID)
Accept task(req ID)
Update status
opt
[accept/reject task]
Task rejected
Assign to another (volunteer ID)
Acess(userID)
Access (user ID)
displayresources (records)
display resources(records)
alt
[Anonymous donation]
Anonymously update record (type,quantity,action)
[else]
update record (type,quantity,action)
update inventory (record details)
requestreport (eventname,date)
Gather, validate data generatereport
display report (data)
save report
request shelter update
alt
[reuse existing shelter]
update existing shelter (ID,availability)
[else]
display sehlter form
add shelter info (location,capacity, availability)
confirmUpdate (shelter info updated)
Alert msg in case of emergency (location)
Alert(location,details)

SYSTEM RISK ANALYSIS

Disaster Management and Relief System is functioning in highly critical and time-sensitive environments where even a failure, wrong data inputs, and/or any kind of delay can lead to worse consequences. Hence, there came a need for a risk analysis that could determine the risk of technology and business types facing the system.

The below tables describe a risk analysis of the system that highlights risks linked to the system and measures to take to ensure that the system is safe with effective measures for disaster response.

TECHNICAL RISKS:

Risk	Type of risk	Probability	Consequences	Mitigation strategy
System crash during peak disaster usage	Product risk	Medium	Victims fail to submit complaint. Failure of the relief system	Load balancing must be done. High quality systems. backups must be installed

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Failure of offline data synchronization	Product risk	Medium	Volunteers do not get the complaints on time. Help delayed	Conflict resolution systems must be installed

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Database overload due to high transaction rate	Product risk	High	Response delayed. Complaint delayed; Help delayed	Use scalable databases and performance optimization

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Incorrect disaster alert generation	Product risk	Medium	Incorrect alerts create unnecessary panic and waste of time and resources	Alerts must be validated properly and source confirmation

SECURITY TYPE RISKS:

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Unauthorized access to victim data	Business risk	Medium	The release of personal information. People will lose trust	Proper system security to protect sensitive information. End to end encryption

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Data tampering in resource records	Product Risk	Low to Medium	Tempering must cause confusion. Problems keeping track of activities	Proper security checks every now and then. Keep track of all the logs and any changes

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Fake volunteer registrations	Business Risk	Medium	Countless Security threats. Misuse of system access	Volunteer activities must be tracked. volunteers must be authenticated from different resources

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Weak password or credential theft	Project Risk	Medium	Anyone can temper the whole system and mess with sensitive data	Proper password policy. Two factor authentication

OPERATIONAL RISKS:

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Volunteers becoming unavailable during emergencies	Project Risk	Medium	Delayed response. People suffer due to insufficient help	Backup options should be available at all times. reassignment systems developed

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Poor coordination between NGOs and volunteers	Business Risk	Medium	Risk of duplicate efforts. Operations will be insufficient at times	Proper coordination between the volunteers and NGO. Centralized tasks

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Incorrect complaint information from victims	Product Risk	Medium	A waste of time and resources. Actual victims will suffer	The location of information must be validated. Proper resources to confirm the complaint

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Human error while updating shelter data	Project Risk	Low	There can be overcrowding of shelters and some shelters may remain unutilized	Data validation rules, admin verification, update history tracking

MANAGEMENT RISKS:

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Human error while updating shelter data	Project Risk	Low	There can be overcrowding of shelters and some shelters may remain unutilized	Data validation rules, admin verification, update history tracking

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Insufficient funding for system maintenance	Business Risk	Medium	This can result in System degradation and outdated features makes the system incompetent	There must be proper Budget planning, sponsorships, phased development

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Legal or compliance issues related to data handling	Business Risk	Low	There will be System shutdown, penalties due to this	The handlers of data must have proper knowledge about protection laws, legal review

Risk	Type of risk	Probability	Consequences	Mitigation strategy
Lack of long-term system ownership	Project Risk	Medium	System abandonment after deployment	Assign responsible authority, maintenance plan

The identified risks and methods for mitigating them reflect a proactive attitude towards the prevention of system failures and inefficiencies. Through the identification of the risks at the design and development stages of the system development life cycle, the system is anticipated to be functioning well under extreme disaster circumstances to ensure timely relief to the affected individuals and coordination between volunteers, NGOs, and donors.

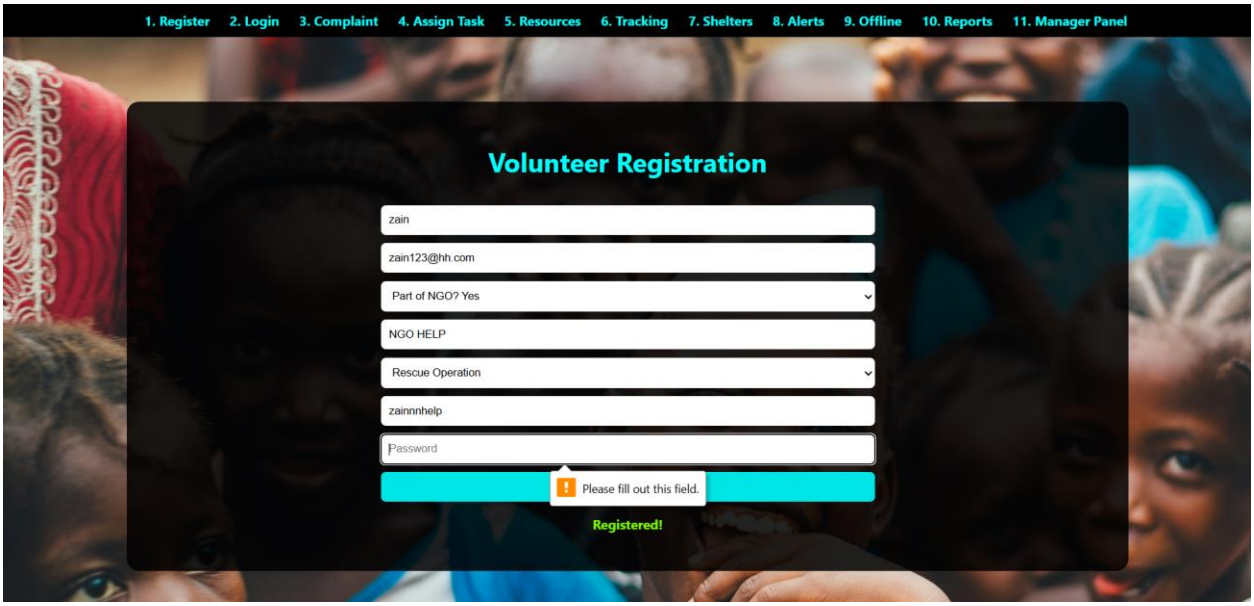
BLACK BOX TESTING

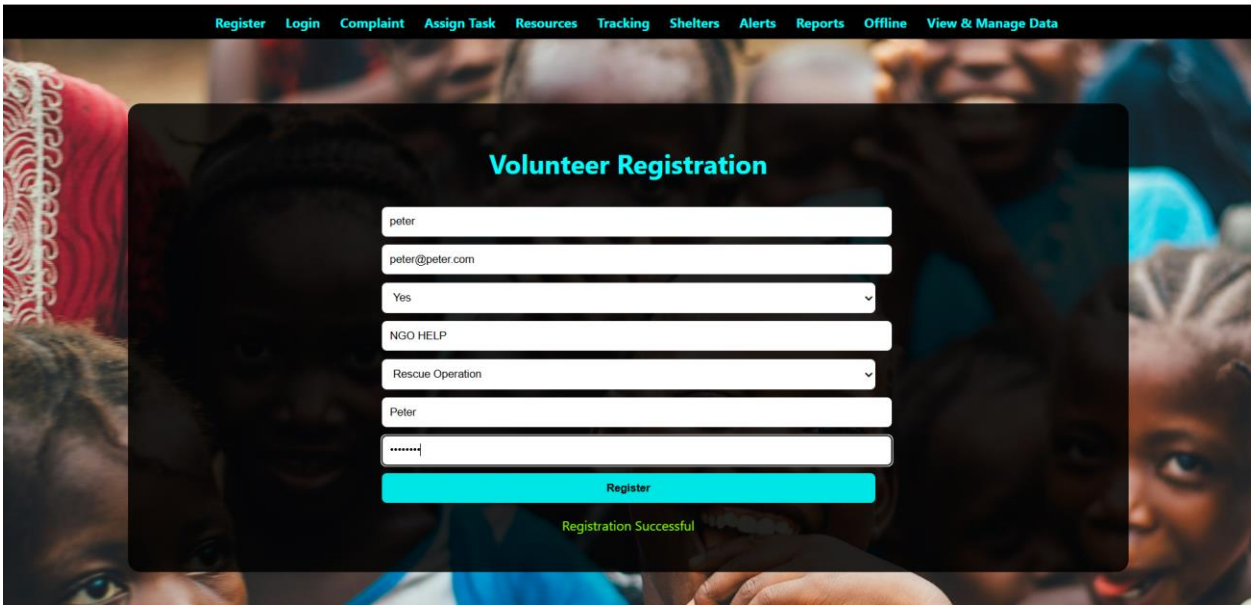
Requirement 1:

Volunteer registration:

- The volunteers must register themselves with their names and personal information.
- The user must add information about their role in the system.
- Every user must have their designated login ID.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Verify successful volunteer registration	Verify validation for incomplete registration
Test ID	BB-01-A	BB-01-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	All required fields filled correctly	One or more required fields left empty
Expected Result	Registration successful message shown	Error message “All required fields must be filled”
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Volunteer Registration
zain
zain123@hh.com
Part of NGO? Yes
NGO HELP
Rescue Operation
zainnnhelp
Password
Please fill out this field.
Registered!

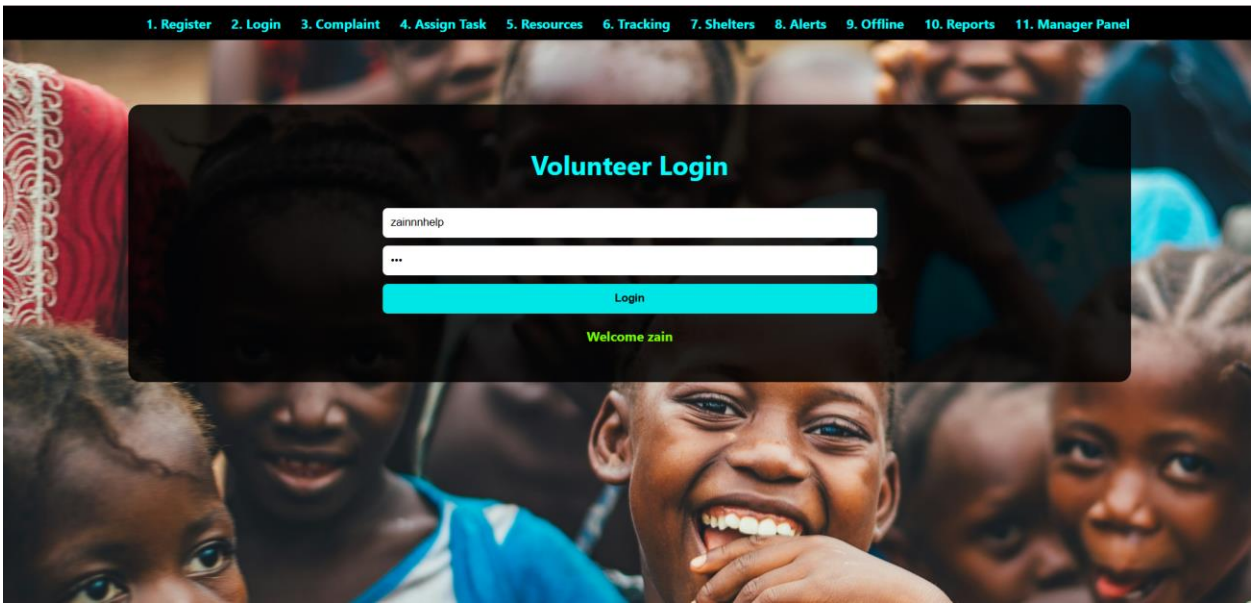

Register
Login
Complaint
Assign Task
Resources
Tracking
Shelters
Alerts
Reports
Offline
View & Manage Data
Volunteer Registration
peter
peter@peter.com
Yes
NGO HELP
Rescue Operation
Peter
Register
Registration Successful

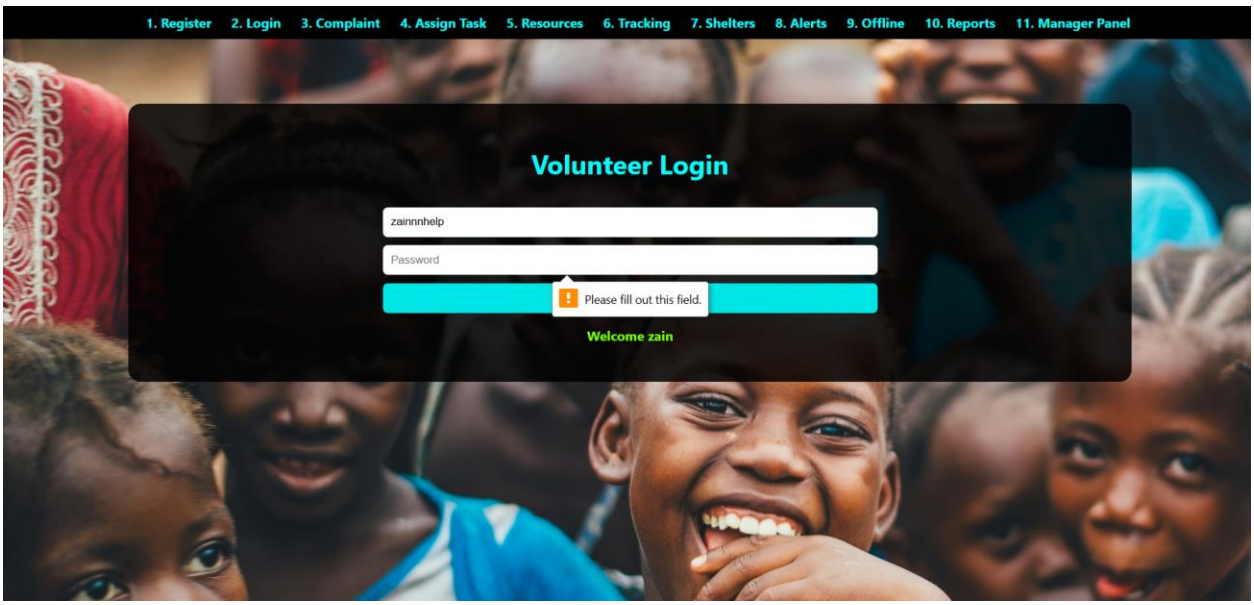
Requirement 2:

2: Login page:

- Login allows users to enter username and password to access the system.
- The system prevents unauthorized access by rejecting invalid login attempts
- Error message are displayed when invalid or missing credentials are entered.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Verify login with correct credentials	Verify login with incorrect credentials
Test ID	BB-02-A	BB-02-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Correct username and password	Incorrect username and password
Expected Result	Login successful message	Error message ‘Invalid credentials’
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Volunteer Login
zainnnhelp
Login
Welcome zain

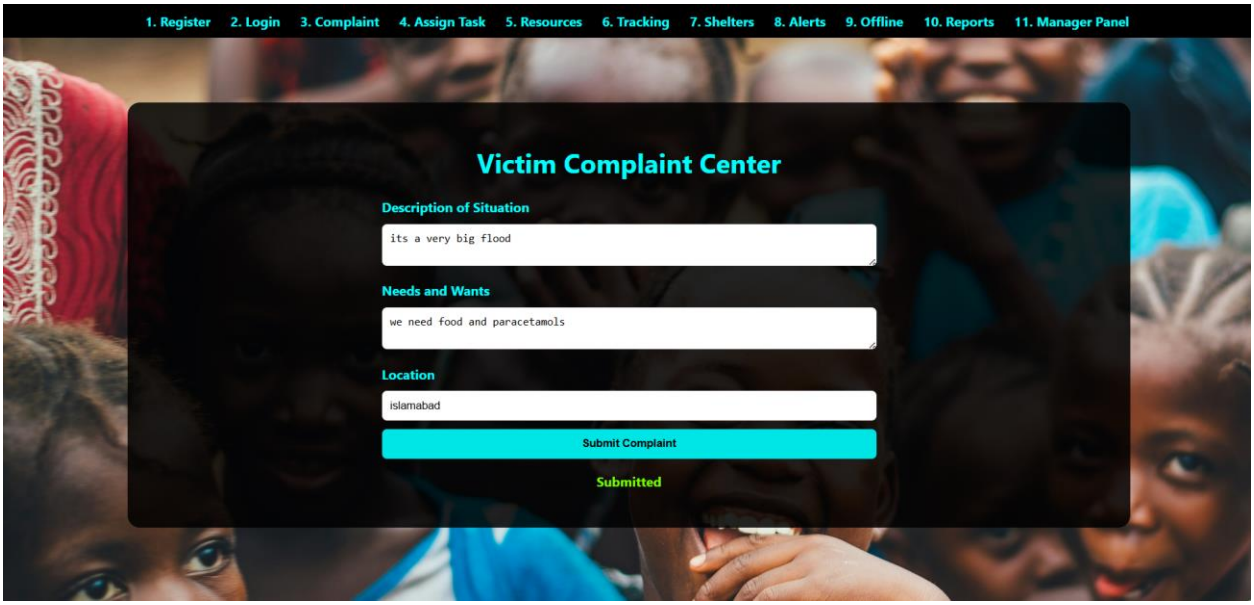

1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Volunteer Login
zainnnhelp
Password
Please fill out this field.
Welcome zain

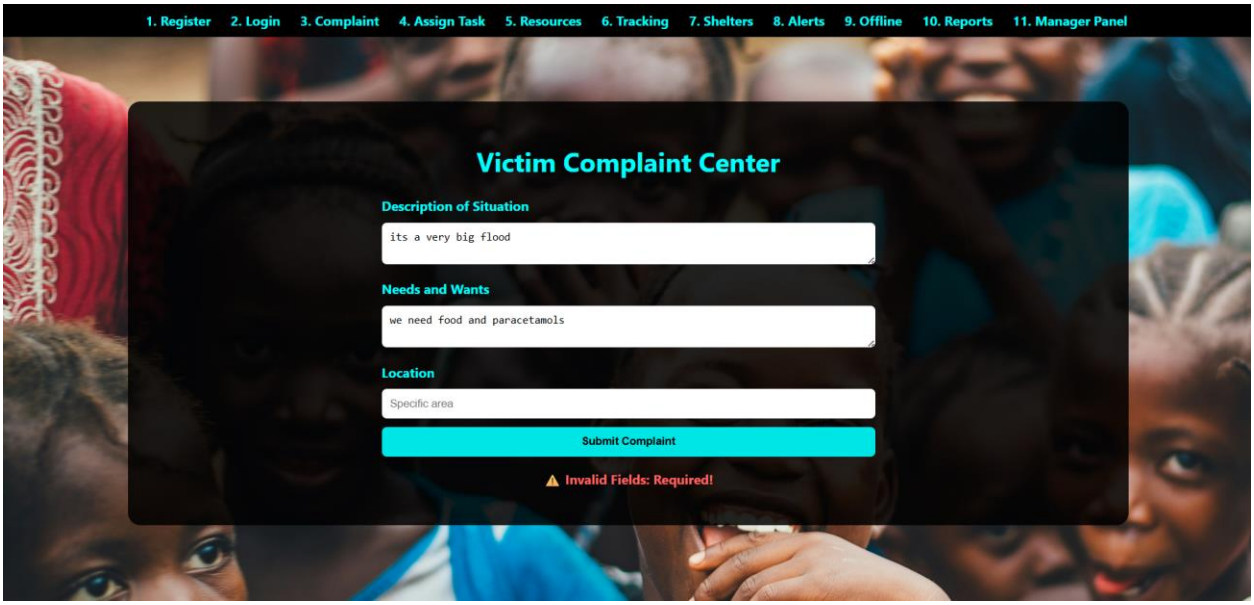
Requirement 3:

3: Complaint management center:

- The user can enter their complaints in the system complaint center.
- The victims can mention their needs and wants in case of any disaster situation.
- The system must receive and process the complaint and forward it to the NGO'S and volunteers for them to take action.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Verify complaint submission	Verify empty complaint submission
Test ID	BB-03-A	BB-03-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Complaint description & location entered	Empty description/location
Expected Result	Complaint submitted successfully	Complaint not submitted / error shown
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Victim Complaint Center
Description of Situation
its a very big flood
Needs and Wants
we need food and paracetamols
Location
islamabad
Submit Complaint
Submitted

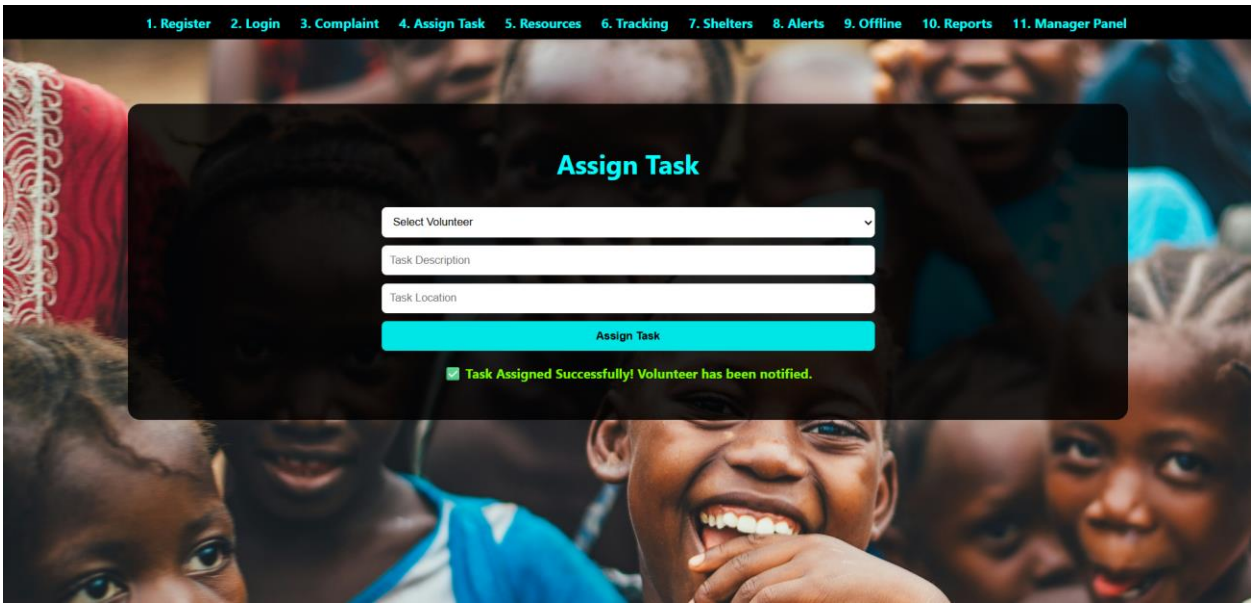

1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Victim Complaint Center
Description of Situation
its a very big flood
Needs and Wants
we need food and paracetamols
Location
Specific area
Submit Complaint
⚠ Invalid Fields: Required!

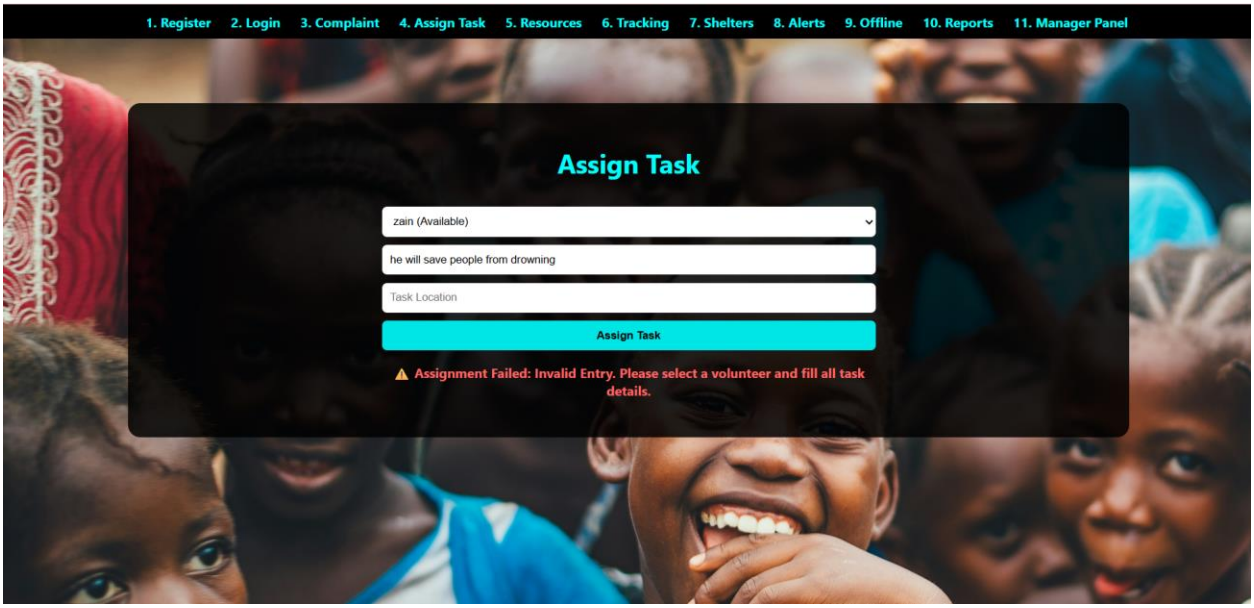
Requirement 4:

4: Assigning tasks to volunteers:

- The complaint of the victim is processed by the system.
- The system shows the complaint on the user's side.
- The system assigns the complaint to the available volunteers by tracking the volunteer activity.
- The volunteers who are assigned the task then take action to resolve the issue.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Assign task to volunteer	Validate missing task info
Test ID	BB-04-A	BB-04-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Volunteer selected + task description	Volunteer not selected or task empty
Expected Result	Task assigned successfully	Error message displayed
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Assign Task
Select Volunteer
Task Description
Task Location
Assign Task
Task Assigned Successfully! Volunteer has been notified.

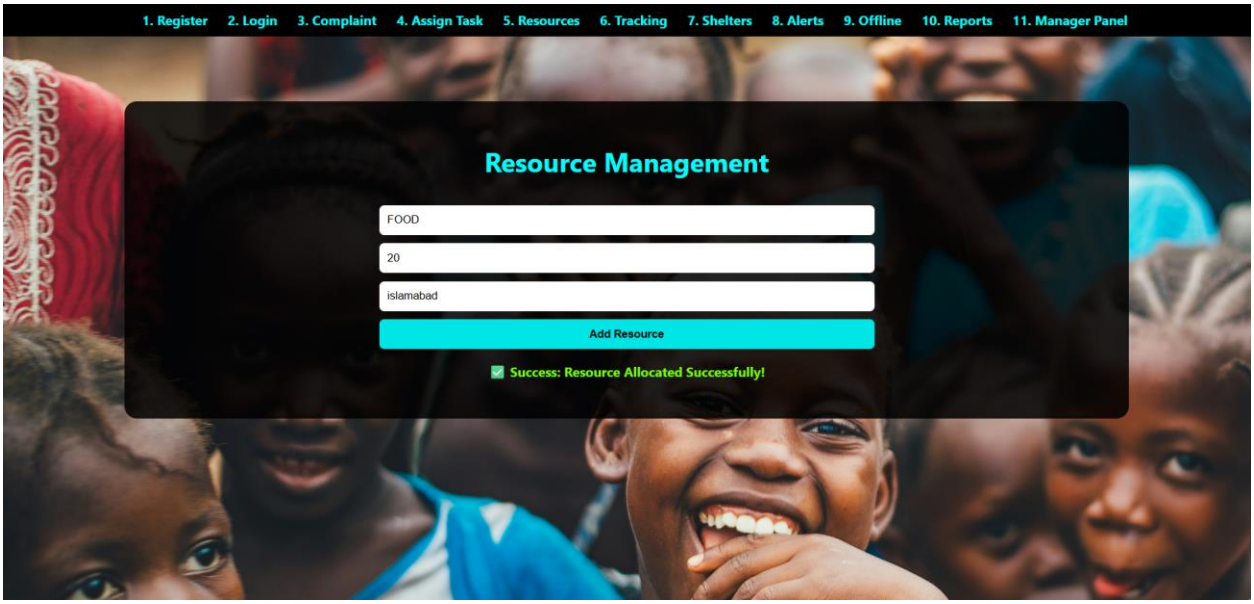

1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Assign Task
zain (Available)
he will save people from drowning
Task Location
Assign Task
Assignment Failed: Invalid Entry. Please select a volunteer and fill all task details.

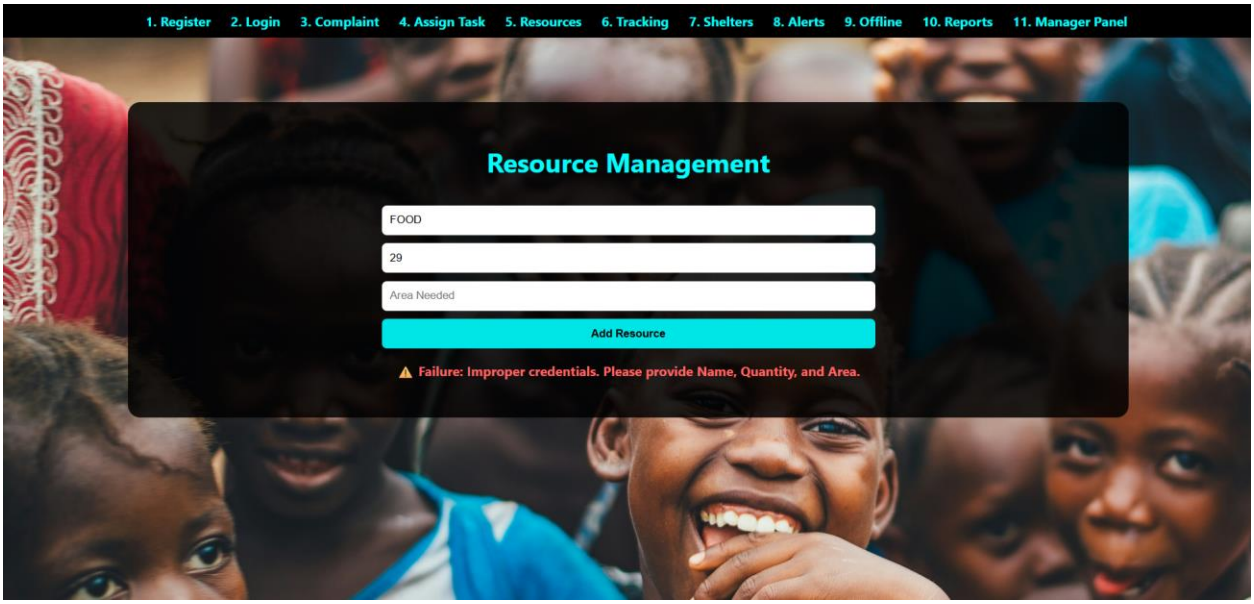
Requirement 5:

5: Management of resources:

- The system keeps track of the resources that are available for the victims.
- The authorized personnel can also alter the records by entering or deleting any extra information.
- They can enter the any updates in the donation about any new donations.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Add new resource	Validate empty resource fields
Test ID	BB-05-A	BB-05-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Resource name & quantity	Empty name or quantity
Expected Result	Resource added successfully	Resource not added
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Resource Management
FOOD
20
islamabad
Add Resource
Success: Resource Allocated Successfully!

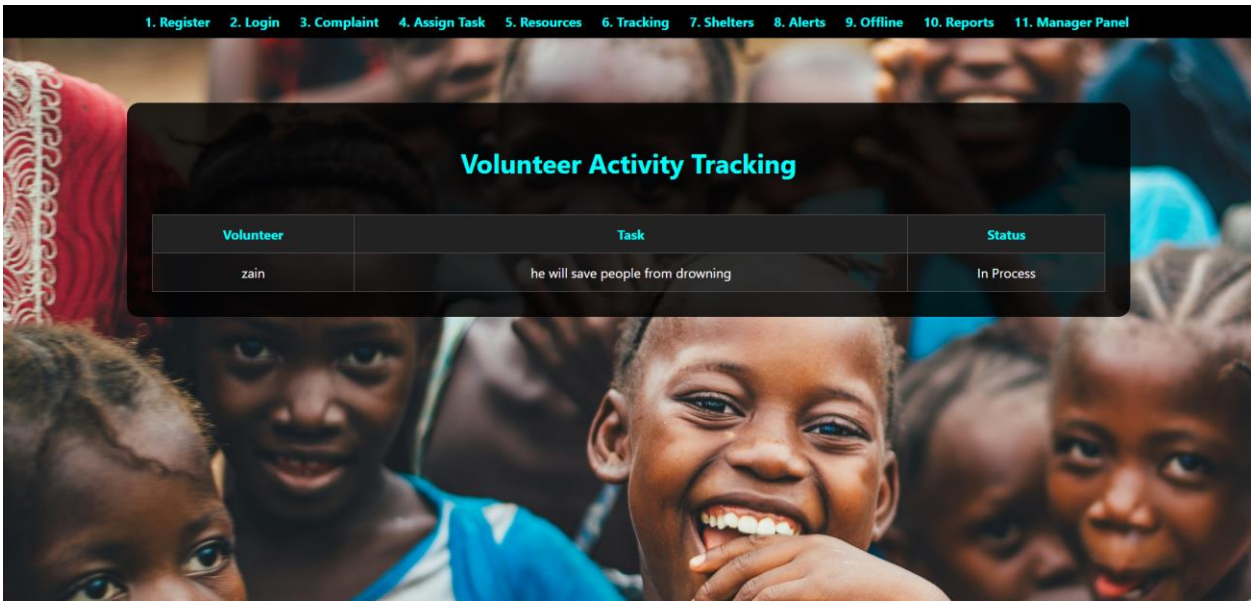

1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Resource Management
FOOD
29
Area Needed
Add Resource
Failure: Improper credentials. Please provide Name, Quantity, and Area.

Requirement 6:

6: Track of volunteer activities:

- The system keeps track of the activities of different volunteers.
- It checks if the volunteers are available.
- It shows what volunteers are assigned to what task at what time.
- The volunteers can add and update their status according to their availability.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Track assigned tasks	No tasks assigned
Test ID	BB-06-A	BB-06-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Assigned task exists	No task data
Expected Result	Tasks displayed correctly	Empty tracking table
Actual Result	Passed	Passed

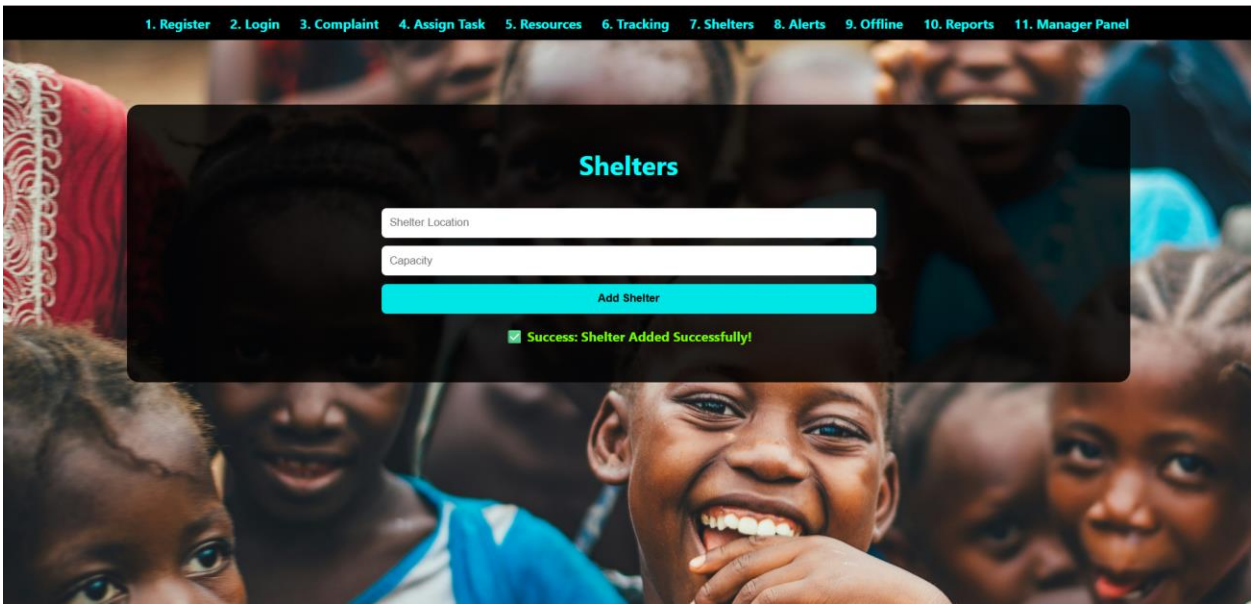


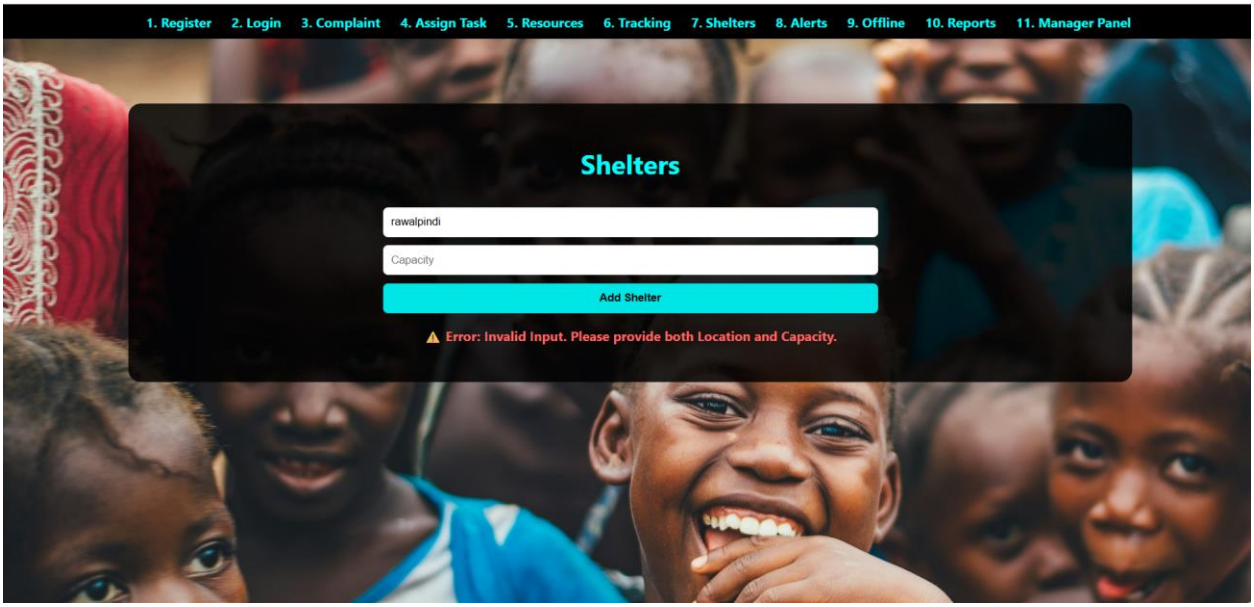
Requirement 7:

7: Checking shelters and relief camps:

- The system keeps a track of the availability of shelter camps for the victims in case of a disaster.
- The volunteers can also update the availability or unavailability of the shelters.
- The system can also keep track of the location of the relief camps
- It can also track the amount of people a relief camp can accommodate.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Add shelter details	Validate missing shelter info
Test ID	BB-07-A	BB-07-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Location & capacity	Empty location or capacity
Expected Result	Shelter added	Shelter not added
Actual Result	Passed	Passed


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Shelters
Shelter Location
Capacity
Add Shelter
Success: Shelter Added Successfully!

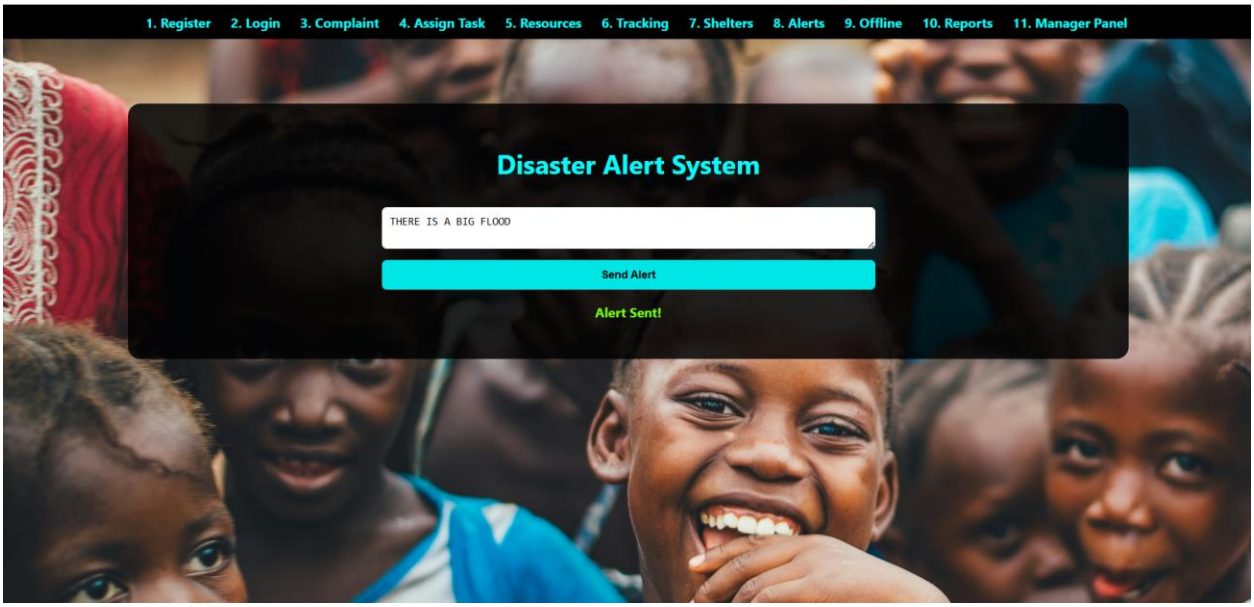

1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Shelters
rawalpindi
Capacity
Add Shelter
Error: Invalid Input. Please provide both Location and Capacity.

Requirement 8:

8: Disaster alert system:

- The system helps the volunteers can get a warning message or an alert in case of any kind of emergency situation.
- This will help the volunteers to take an immediate action upon the situation and prepare necessary reliefs for the affected people.
- The alerts are traced so the system can inform the ngo/volunteers about the location.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Send disaster alert	Send empty alert
Test ID	BB-08-A	BB-08-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Alert message entered	No alert message
Expected Result	Alert sent to volunteers	Alert not sent
Actual Result	Passed	Passed

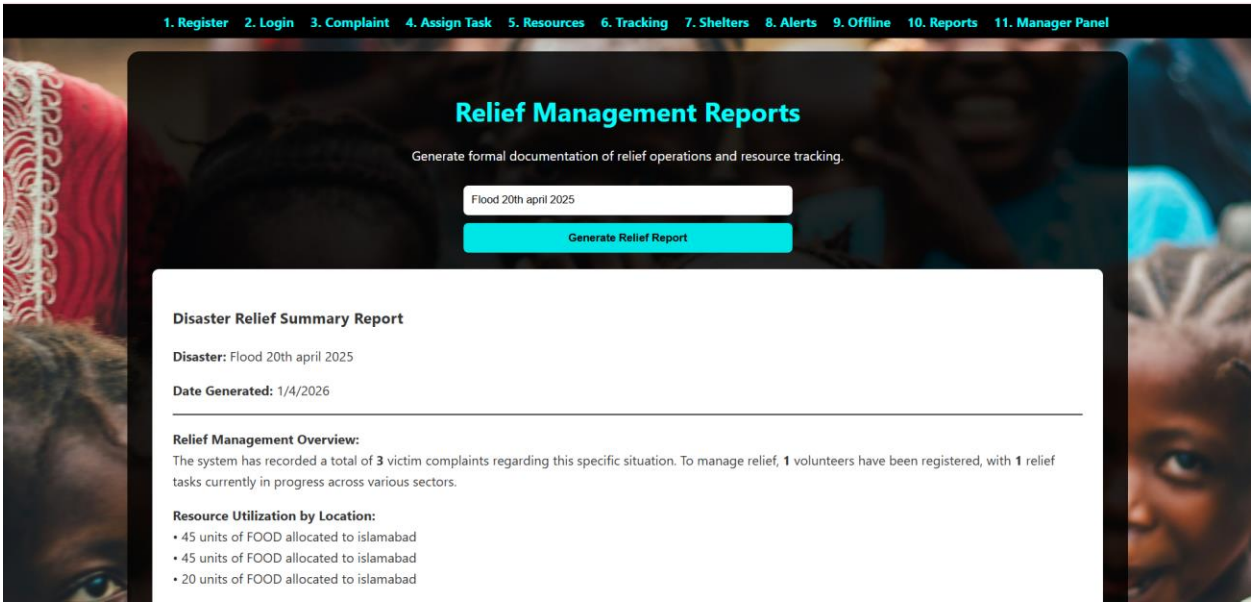


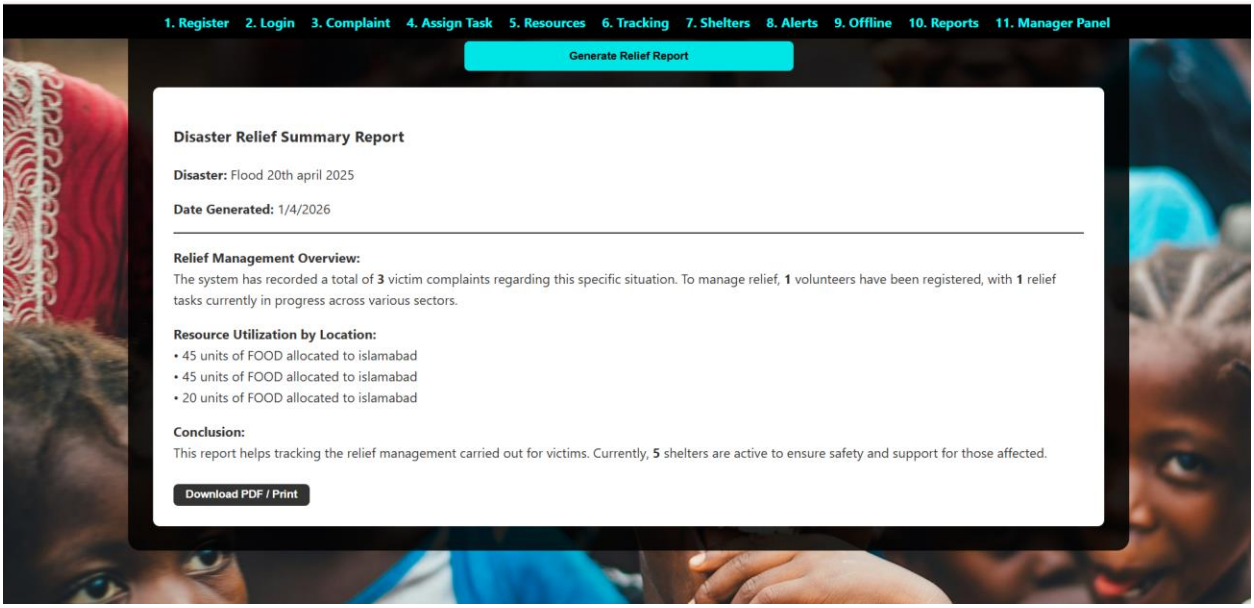
Requirement 9:

9: Generation of reports:

- The system is able generated well written reports for the user.
- These reports may contain information about a specific disaster.
- These reports can be about the number of resources used in a specific situation or at a specific location.
- These reports help the users to keep track of the relief management they carried out for the victims.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Verify report generation when data exists	Verify report generation when no data exists
Test ID	BB-09-A	BB-09-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	User clicks **"Generate Summary"** button	User clicks **"Generate Summary"** with no records stored
Expected Result	System generates summary showing number of users, complaints, resources, shelters, and tasks	System generates report with zero values instead of crashing
Actual Result	Passed	Passed


1. Register 2. Login 3. Complaint 4. Assign Task 5. Resources 6. Tracking 7. Shelters 8. Alerts 9. Offline 10. Reports 11. Manager Panel
Relief Management Reports
Generate formal documentation of relief operations and resource tracking.
Flood 20th april 2025
Generate Relief Report
Disaster Relief Summary Report
Disaster: Flood 20th april 2025
Date Generated: 1/4/2026
Relief Management Overview:
The system has recorded a total of 3 victim complaints regarding this specific situation. To manage relief, 1 volunteers have been registered, with 1 relief tasks currently in progress across various sectors.
Resource Utilization by Location:
• 45 units of FOOD allocated to islamabad
• 45 units of FOOD allocated to islamabad
• 20 units of FOOD allocated to islamabad

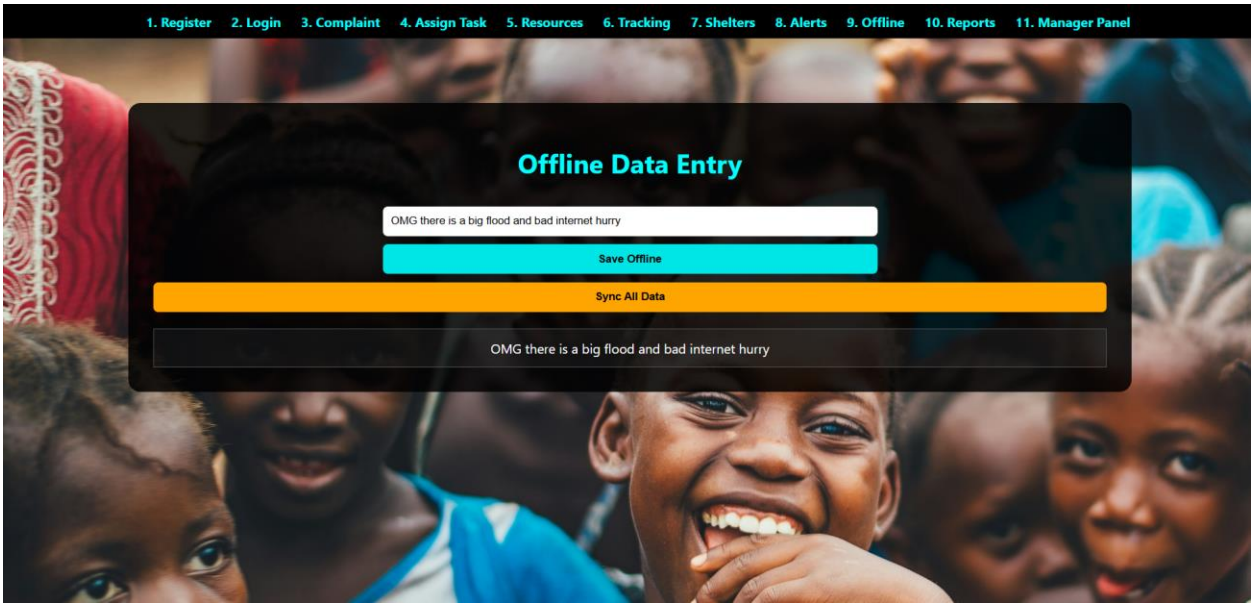

1. Register 2. Login 3. Complaint 4. Assign Task 5. Resources 6. Tracking 7. Shelters 8. Alerts 9. Offline 10. Reports 11. Manager Panel
Generate Relief Report
Disaster Relief Summary Report
Disaster: Flood 20th april 2025
Date Generated: 1/4/2026
Relief Management Overview:
The system has recorded a total of 3 victim complaints regarding this specific situation. To manage relief, 1 volunteers have been registered, with 1 relief tasks currently in progress across various sectors.
Resource Utilization by Location:
• 45 units of FOOD allocated to islamabad
• 45 units of FOOD allocated to islamabad
• 20 units of FOOD allocated to islamabad
Conclusion:
This report helps tracking the relief management carried out for victims. Currently, 5 shelters are active to ensure safety and support for those affected.
Download PDF / Print

Requirement 10:

10: Handling system without internet:

- This system helps manage and helps enter the information in the system without the need of internet.
- The system and the information will synchronize as soon as the system catches any signal of internet.

Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Save offline data	Validate empty offline input
Test ID	BB-10-A	BB-10-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Offline complaint entered	Empty input
Expected Result	Data saved offline	Error message shown
Actual Result	Passed	Passed



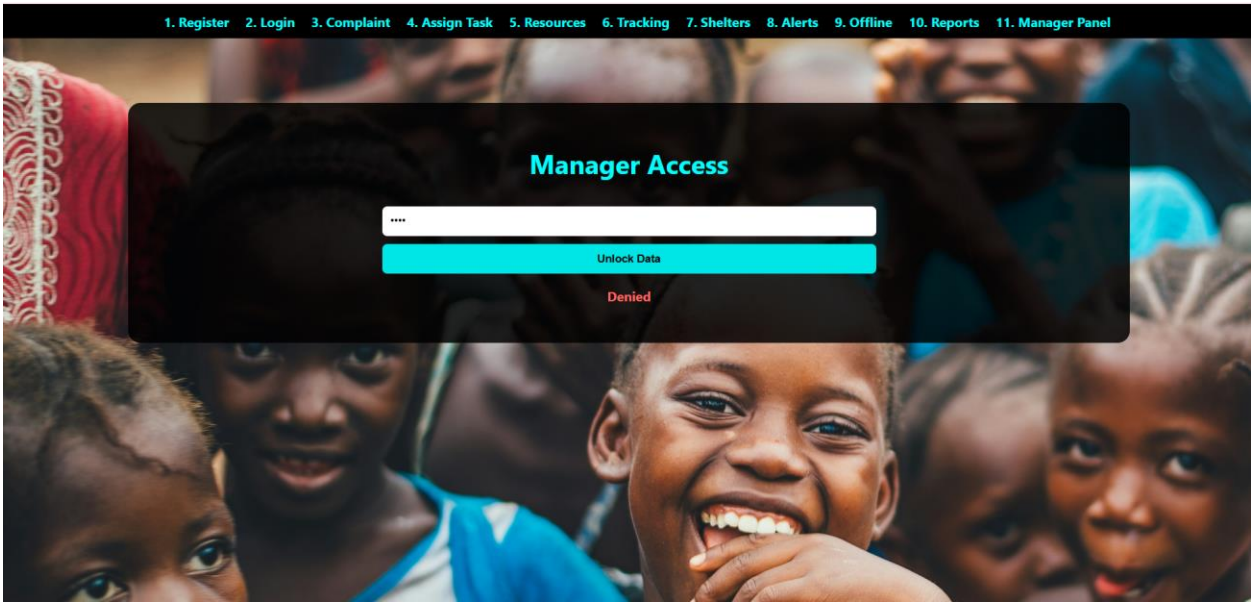
Requirement 11:

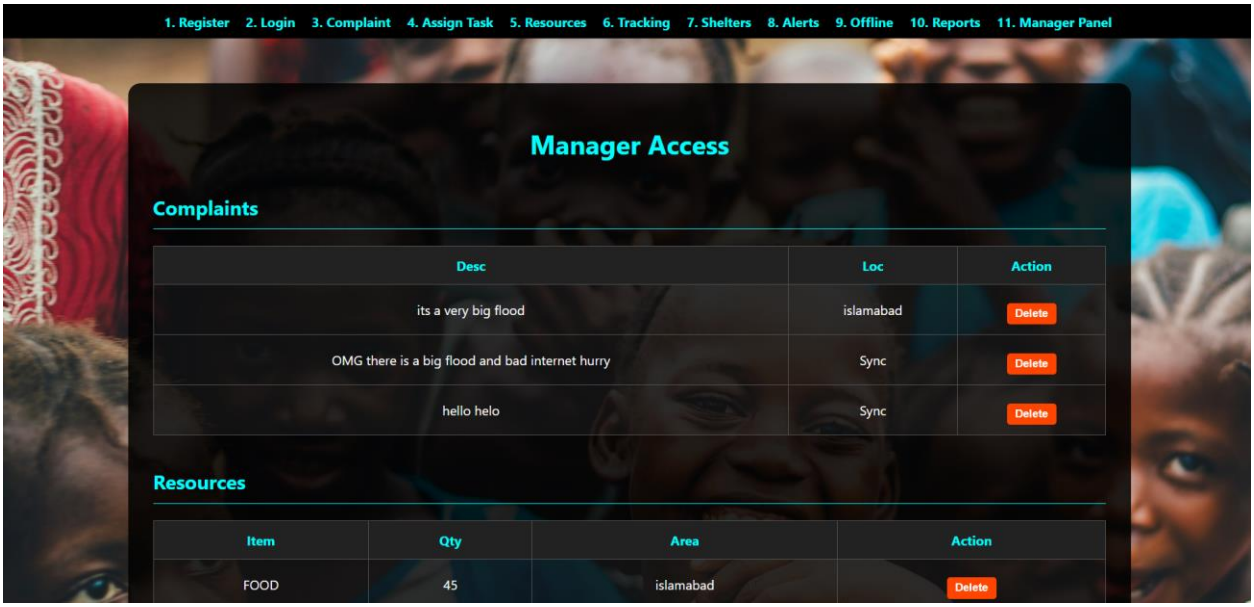
11: Security:

- The system must provide full confidentiality to the victims and members.
- The donors must not be able to access any information about the victims.
- Only the managers must be able to view everything about the victims, donors, NGO's etc.

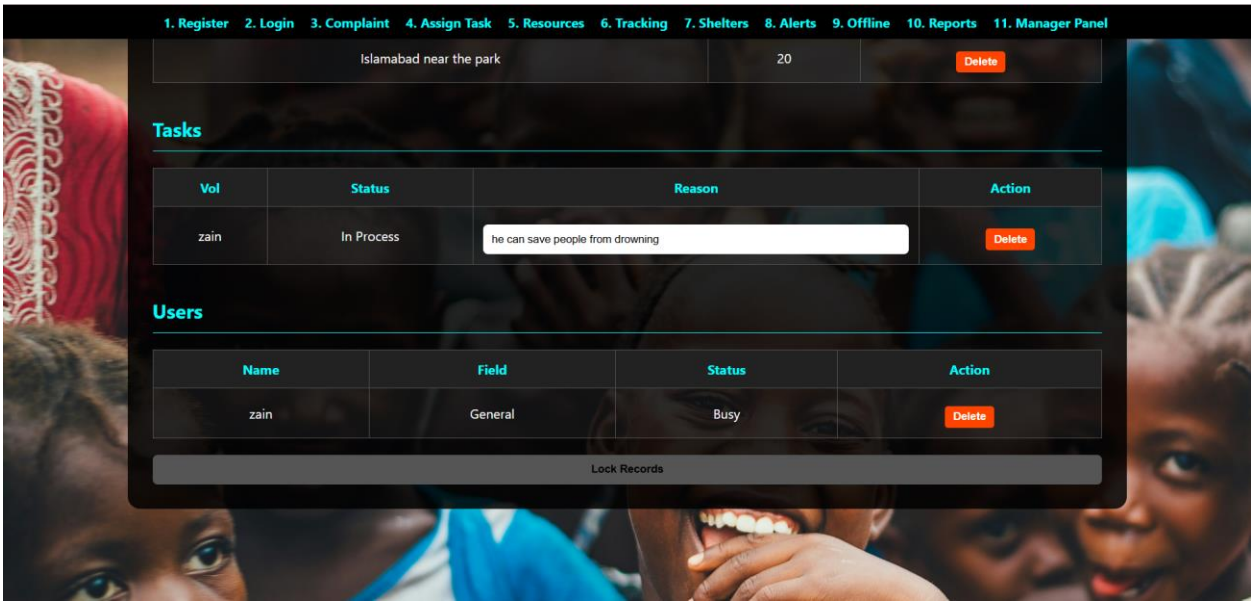
Field	TC-01 (Valid Input)	TC-02 (Invalid Input)
Date	10-12-25	10-12-25
System	Disaster management system	Disaster management system
Objective	Restrict unauthorized access	Prevent donor from viewing victim data
Test ID	BB-11-A	BB-11-B
Version	1	1
Test type	Black Box Testing	Black Box Testing
Input	Authorized user access	Unauthorized user access
Expected Result	Data visible	Access denied
Actual Result	Passed	Passed

Password was admin123 but we entered 222:






1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Resources
Item
Qty
Area
Action
FOOD
45
islamabad
Delete
FOOD
20
islamabad
Delete
Shelters
Loc
Cap
Action
islamabad
20
Delete
Islamabad near the park
20
Delete
Tasks


1. Register
2. Login
3. Complaint
4. Assign Task
5. Resources
6. Tracking
7. Shelters
8. Alerts
9. Offline
10. Reports
11. Manager Panel
Islamabad near the park
20
Delete
Tasks
Vol
Status
Reason
Action
zain
In Process
he can save people from drowning
Delete
Users
Name
Field
Status
Action
zain
General
Busy
Delete
Lock Records

SYSTEM OVERVIEW

The Disaster Management and Relief System is a centralized software solution designed to support disaster response by connecting victims, volunteers, NGOs, donors, and administrators through a single platform. The system aims to improve coordination, reduce response time, and ensure transparency during emergency situations.

Victims can submit complaints and help requests, which are processed and assigned to available volunteers based on location and availability. The system also manages resources, shelters, and relief camps, while tracking volunteer activities to maintain accountability. An alert mechanism notifies volunteers during emergencies to enable prompt action.

The system supports offline data entry with automatic synchronization once internet connectivity is restored. Role-based access control and security measures ensure confidentiality of sensitive information. Overall, the system provides a reliable and organized approach to managing disaster relief operations.

FUTURE ENHANCEMENTS

Although the proposed Disaster Management and Relief System fulfills the core functional and non-functional requirements, several enhancements can be implemented to improve its effectiveness and usability further.

In the future, the system can be extended to support mobile applications to allow victims and volunteers to access services more easily during emergencies. GPS-based location tracking can be added to improve the accuracy of complaint locations and volunteer assignments. The system may also integrate real-time disaster prediction and alert mechanisms using external data sources to provide early warnings.

Additional enhancements include multi-language support to improve accessibility for diverse users, advanced reporting and analytics dashboards for better administrative decision-making, and integration with government or emergency service databases to ensure faster coordination. Security features such as enhanced authentication methods can also be strengthened to further protect sensitive data.

These future improvements would increase the system’s reliability, scalability, and real-world applicability, making it more effective for large-scale disaster management operations.

CONCLUSION

The Disaster Management and Relief System was designed to provide an efficient and structured solution for managing disaster response activities. The system successfully integrates victims, volunteers, NGOs, donors, and administrators to ensure timely communication, task coordination, and effective use of resources during emergencies.

All functional and non-functional requirements were carefully addressed, including complaint handling, task assignment, resource management, security, reliability, and availability. Risk analysis identified potential technical, security, and operational risks, while black box testing confirmed that system functionalities perform as expected under both valid and invalid conditions.

In conclusion, the proposed system offers a reliable and scalable approach to disaster management. With further development and real-world implementation, it can significantly enhance relief coordination and help ensure that assistance reaches affected individuals promptly.